Agente Conversacional para E-Commerce: "Patitas pa' que las quiero"

Institución: Universidad Nacional Autónoma de México (UNAM) — Facultad de Ingeniería

Asignatura: Inteligencia Artificial — Práctica 2 (AIML)

Profesor: Dr. José Fidel Urquiza Yllescas

Autora: María Fernanda Ordóñez Figueroa

Dominio elegido: 5.1 Sector E-Commerce — "Dudas repetitivas" (logística y pagos de una tienda de calzado deportivo)

---

1. Resumen y caso de negocio

El proyecto implementa un chatbot de primera línea para la tienda de calzado deportivo **"Patitas pa' que las quiero"**, de **dominio cerrado**, con **AIML** el cual es de respuestas deterministas y trazables. Este resuelve sin intervención humana las dudas que saturan el chat en vivo:

- Hace rastreo de pedidos, retrasos, cambio de dirección y cancelaciones.
- Responde sobre envíos a provincia, todo México, tiempos y costos de envío, paqueterías, pickup en showroom.
- Menciona las formas de pago, meses sin intereses, OXXO, SPEI, PayPal / Mercado Pago, pagos rechazados, facturación.
- Informa sobre cambios de talla, devoluciones, garantía y reembolsos; catálogo, precios, promociones y horarios.

Cuando el bot no puede ayudar, escala a una persona (correo y horario de atención), esto solo despues de tres intentos del usuario, si es que no entiende el bot, lo envia a soporte humano.

---

2. Arquitectura

Código modular dentro de `archivos_aiml/` (se evita llamarlo `aiml/` para no chocar con el paquete Python `aiml`). `startup.aiml` ensambla todos los módulos con `<learn>`.

```
AgenteIAML/
```

```
├── main.py #master

├── normalizar.py

├── probar.py # Batería de pruebas automática contra el bot

├── requirements.txt

├── pruebas/casos.py # Casos de una entrada y conversaciones de varios turnos para automatizar el chequeo en diferentes iteraciones

└── archivos_aiml/

├── startup.aiml

├── negocio.aiml

├── saludos.aiml

├── despedidas.aiml

├── respGenerales.aiml

├── conectores.aiml

├── adverbios.aiml

├── pedidos.aiml

├── envios.aiml

├── ubicacion.aiml

├── horarios.aiml

├── pagos.aiml

├── facturacion.aiml

├── promos.aiml

├── precios.aiml

├── productos.aiml

├── devoluciones.aiml

├── soporte.aiml
```

```
└── fallback.aiml
```

Diseño

Cada intención tiene **una sola** categoría canónica (`INTENCION RASTREO` , `INTENCION MSI` , …). Todas las formas de preguntarla son categorías de una línea que hacen `<srai>` hacia ella, y cambiar un texto se hace en un solo lugar. Los datos que cambian con el negocio (precios, plazos, URL, correo) viven en `negocio.aiml` y las plantillas los consultan con `<srai>INFO ...</srai>` .

Como AIML 2.0 no tiene el comodín de "cero o más palabras", cada frase clave se registra en 4 variantes (`P` , `P *` , `* P` , `* P *`), de modo que la frase se reconozca al inicio, en medio o al final de la oración, si no, no entra en varios casos y rompe con la fluides de la conversacion

El reconocimiento se hace en capas:

1. **Reducción (normalización):** `conectores.aiml` y `adverbios.aiml` descartan relleno al inicio/final ("oye, disculpa, quisiera saber si…", "por favor") y reprocesan el resto con `<srai><star/></srai>` / `<sr/>` , esto lo vi de un repositorio de github para otro bot.
2. **Intención específica:** frases exactas, frases con hueco ("DONDE * PEDIDO") y palabras clave.
3. **Contexto:** `<that>` (sí/no tras una pregunta del bot) y `<topic>` (el mismo "¿cuánto tarda?" significa algo distinto en ENVIOS, PAGOS o CAMBIOS).
4. **Rescate por palabra clave:** si nada reconoció el mensaje, el comodín `*` lo pasa por `RESCATE <mensaje>` , que detecta una palabra clave (pedido, pago, cambio, estados de la república…) y muestra el menú del tema.
5. **Fallback escalonado:** contador de fallos con `<condition>` : 1.º orienta, 2.º da ejemplos, 3.º canaliza a una persona.

---

3. Etiquetas AIML empleadas

- **`<category>` , `<pattern>` , `<template>` :** unidad básica de conocimiento.
- **`<srai>` / `<sr/>` :** redirección canónica (sinónimos → una respuesta) y normalización por reducción.
- **Comodín `*` y `<star/>` :** captura de nombre, número de guía y relleno de la oración.
- **`<that>` :** flujos sí/no ("¿ya tienes tu número de guía?") y captura del nombre/guía solo cuando el bot acaba de pedirlos.
- **`<topic>` :** contexto de CAMBIOS (REQUISITOS / INICIAR CAMBIO / SALIR), FACTURA, PAGOS, ENVIOS, RASTREO y PRODUCTOS.
- **`<set>` / `<get>` :** memoria de sesión: `usuario` , `num_pedido` , `topic` , `fallos` .

- `<condition>`: saludar por nombre si ya se conoce; rastreo distinto si ya hay guía; contador de fallos.
- `<think>`: guarda variables sin imprimirlas.
- `<formal>`: pone el nombre en formato "Maria Fernanda".
- `<random>`: variación en saludos, despedidas y confirmaciones.
- `<learn>`: carga modular desde `startup.aiml`.

---

4. Requisitos e instalación

- Python 3.8 o superior
- Paquete `python-aiml` (intérprete AIML), que se instala con el txt que añadi para evitar problemas de versiones

```
cd AgenteIAML

python3 -m venv venv #entorno virtual como opcional, a mi parecer buena practica

source venv/bin/activate # activacion del entorno virtual, si se creo

pip install -r requirements.txt # equivale a: pip install python-aiml
```

5. Ejecución y pruebas

```
python main.py # conversar con el bot (escribe "salir" para terminar)

python probar.py # batería automática: casos de una entrada + conversaciones con contexto
```

`probar.py` termina con código 0 si todas las pruebas pasan. y si la prueba fue exitosa, se obtendra un "Conversaciones con contexto: 12/12"

Casos de prueba sugeridos

| Intención / módulo | Entrada del usuario | Comportamiento esperado |

| --- | --- | --- |

| Bienvenida y nombre | `hola` → `me llamo Fernando` | Pide el nombre; lo guarda con `<set>` y responde "Mucho gusto, Fernando". |

| Persistencia | `hola` (otra vez) / `¿cómo me llamo?` | Saluda por su nombre con `<condition>` / `<get>`. |

| Rastreo con contexto | `¿Dónde está mi paquete?` → `sí` → `GUIA98765` | `<that>` interpreta el "sí", se guarda la guía y se muestra el estatus. |

| Rastreo, ramal "no" | `como rastreo mi pedido` → `no la tengo` | Explica dónde encontrar la guía. |

| Tema CAMBIOS | `política de cambios` → `requisitos` → `iniciar` → `salir` | Dentro de `<topic>` CAMBIOS, las respuestas cortas se interpretan en contexto. |

| Mismo mensaje, otro tema | `aceptan oxxo` → `y cuánto tarda` / `cuánto tarda mi envío` | La respuesta depende del `<topic>` activo. |

| Envíos a provincia | `¿envían a Oaxaca?`, `mandan a yucatan` | Confirma cobertura nacional (también por nombre de estado/ciudad). |

| Pagos | `¿hay msi?`, `se puede pagar en una tienda de conveniencia` | MSI / OXXO con los datos de `negocio.aiml`. |

| Frases largas con relleno | `oye una pregunta, cuanto me cobran de envio a guadalajara` | La capa de reducción quita el relleno y se responde el costo. |

| Fuera de dominio | `¿Qué opinas de Trump?` / `asdf` | Opinión: redirige al dominio. Texto sin sentido: fallback escalonado. |

---

6. Dificultades técnicas encontradas y soluciones

1. **El comodín `*` exige al menos una palabra.** Patrones como `* TARJETA *` no reconocían `tarjeta` (sola), `pagar` ni `hay msi`.

Solución: registrar cada frase clave en las 4 variantes (`P`, `P *`, `* P`, `* P *`).

2. **Prioridad entre patrones.** AIML elige primero la palabra literal y solo después el comodín, así que una regla de relleno al inicio ("DONDE *", "MI *") le ganaba a una intención con la palabra clave en medio de la frase.

Solución: reducción por capas relleno → intención, que son las frases con hueco "DONDE * PEDIDO" y sustantivos genéricos que no roban prioridad a intenciones más específicas.

3. **Sí / No ambiguos (contexto).** El "sí/no" tras "¿tienes tu número de guía?" caía al fallback. *Solución:* `<that>`. Como `python-aiml` compara con la **respuesta completa** del bot y la especificación con la **última oración**, cada `<that>` se registra en las dos variantes (`PROMPT` y `* PROMPT`). Las preguntas de seguimiento se redactaron **sin** `¿ ¡ « »` porque el intérprete solo limpia la puntuación ASCII.

4. **Tildes y signos de puntuación.** `python-aiml` no quita tildes ni `¿` / `¡`. *Solución:* `normalizar.py` (`unicodedata`) deja la entrada en MAYÚSCULAS sin tildes ni signos; todos los patrones están escritos igual. Limitación: un nombre con tilde ("María") se guarda como "Maria".
5. **`<set>` imprime su valor.** Respuestas como `CAMBIOS Cuentas con 30 días…` o `222323 Hemos consultado…` salían con la variable pegada. *Solución:* envolver todo `<set>` en `<think>`.
6. **Todo "sí" / "no" y "soy …" capturaban nombres o guías.** *Solución:* la captura del nombre y de la guía con `*` solo se activa cuando el bot acaba de pedirlos (`<that>`); "no tengo / no la encuentro" se interpreta como respuesta negativa.
7. **Mensajes fuera de dominio o con palabras no previstas.**

Solución: fallback de dos etapas el rescate por palabra clave + escalamiento a persona que es soporte tras tres fallos seguidos.

Evaluación

`probar.py` ejecuta más de 400 mensajes escritos como los escribiría un usuario real (con tildes, signos, errores de dedo y relleno) y 12 conversaciones de varios turnos. Los casos se fueron agregando en rondas: cada conjunto nuevo de frases no vistas servía primero como medición de generalización y después se corregían las fallas con reglas generales.

---

7. Licencia y créditos

Software desarrollado con fines académicos bajo la licencia **GNU General Public License v3.0 (GPL-3.0)** para la asignatura de Inteligencia Artificial, Facultad de Ingeniería, UNAM.

Referencia principal: https://github.com/MyRobotLab/pyrobotlab/tree/master